

VELLORE INSTITUTE OF TECHNOLOGY

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

VITyarthi - Build Your Own Project Evaluation

COURSE PROJECT REPORT

SmartCampus 360

Integrated Academic & Resource Allocation System

Student Name: **YATHAM JATHINDRA REDDY**

Registration No: **25BAI10611**

Program of Study: **B.Tech Artificial Intelligence & Machine Learning**

Course Code & Title: **CSE2006 — Programming in Java**

Semester / Term: **Fall Semester 2025–2026**

Submission Date: **September 16, 2026**

Submitted in partial fulfillment of the academic requirements for the course CSE2006 (Programming in Java)

2. Introduction

Higher education academic operations represent a complex orchestration of student preferences, faculty loads, time slot constraints, prerequisite validations, and room capacities. In institutions utilizing modern academic models such as the Fully Flexible Credit System (FFCS), students select their desired courses, faculty members, and scheduling slots dynamically. Without strict algorithmic governance, issues like timetable slot clashes, credit limit exceedance, and inefficient section queueing can emerge.

SmartCampus 360 is a high-performance, modular academic management engine engineered entirely in Java 21. It models academic entities (students, faculty, administrators, theory lectures, and computer laboratories) into a cohesive Object-Oriented architecture. The project demonstrates real-time slot conflict resolution, credit regulation, fair FIFO waitlisting, dynamic 10-point CGPA recalculation, and background multithreaded notification dispatching.

3. Problem Statement

University enrollment and student management systems face systemic operational hurdles:

- **Timetable Slot Collisions:** In complex timetables (e.g. morning/evening shifts with compound laboratory blocks), students often register for overlapping classes.
- **Manual Waitlist Processing:** When popular courses reach seat limits, students are forced to continually refresh web pages. Vacant seats resulting from drops often go unutilized.
- **Credit Overload:** Lack of atomic validation allows students to exceed academic credit caps (27 credits/semester), complicating degree auditing.
- **Attendance Debarment Risk:** Failure to provide real-time attendance warnings leads to surprise debarments when students drop below the mandatory 75% threshold.

4. Functional Requirements

Module	Feature Code	Description
1. User & Auth	FR-1.1 FR-1.2	Role-Based Access Control (RBAC) supporting Student, Faculty, and Admin profiles with SHA-256 password hashing.
2. Course Catalog	FR-2.1 FR-2.2	Management of Theory courses and Laboratory courses with specific slots, credits, and capacity limits.
3. Registration Engine	FR-3.1 FR-3.2 FR-3.3	Automated prerequisite verification, credit limit enforcement (max 27), and deterministic timetable slot collision checking.
4. Waitlist Engine	FR-4.1 FR-4.2	Automatic FIFO waitlist queuing when capacity is reached; instant promotion of the next student when a seat is dropped.

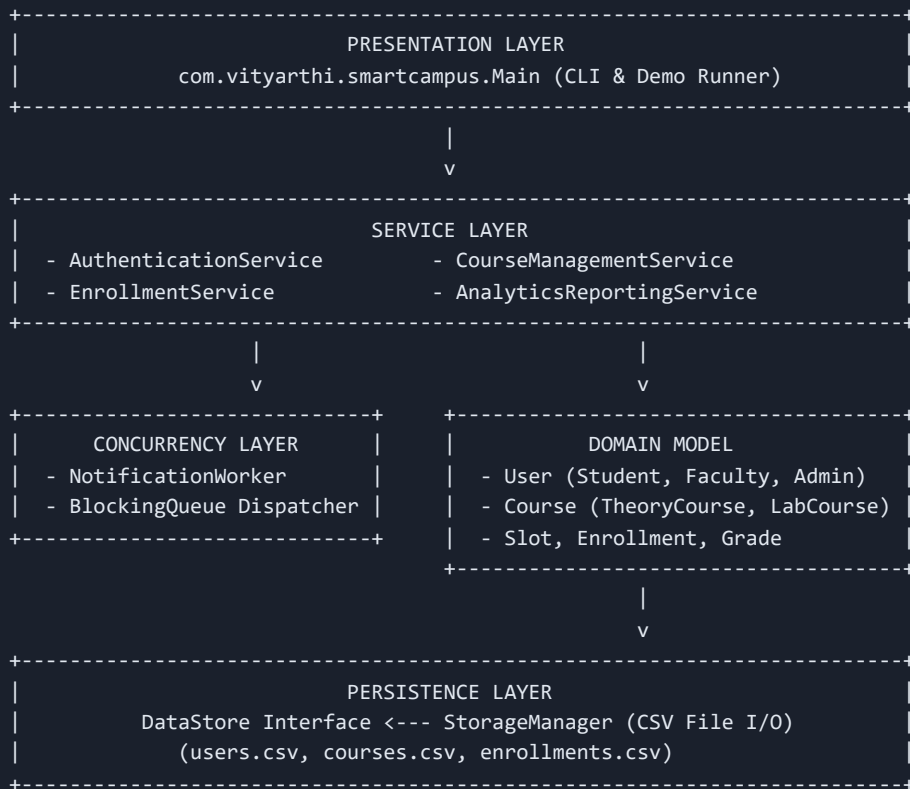
Module	Feature Code	Description
5. Academic & Attendance	FR-5.1	Dynamic 10-point CGPA computation weighted by credits, and real-time attendance tracking flagging students below 75%.
	FR-5.2	

5. Non-Functional Requirements

- **Performance:** In-memory lookups via `ConcurrentHashMap` execute in $O(1)$ time complexity. Conflict detection runs in $O(K)$ where K is current registrations.
- **Security:** Clear separation of roles via RBAC. Sensitive data and credentials are kept encapsulated and hashed.
- **Reliability & Persistence:** Persistent state synchronization through CSV file I/O ensuring data durability across system reboots.
- **Concurrency & Thread Safety:** Non-blocking background event dispatching using `BlockingQueue` and daemon threads.
- **Maintainability:** Clean architecture strictly adhering to SOLID principles and package modularity.

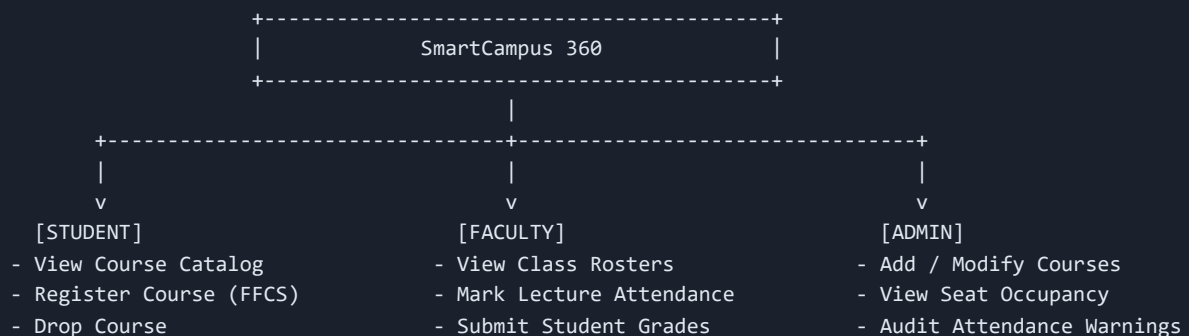
6. System Architecture

The system follows a multi-tiered architecture decoupling user presentation, business logic services, concurrency workers, domain entities, and file-based data stores:



7. Design Diagrams

7.1 Use Case Diagram



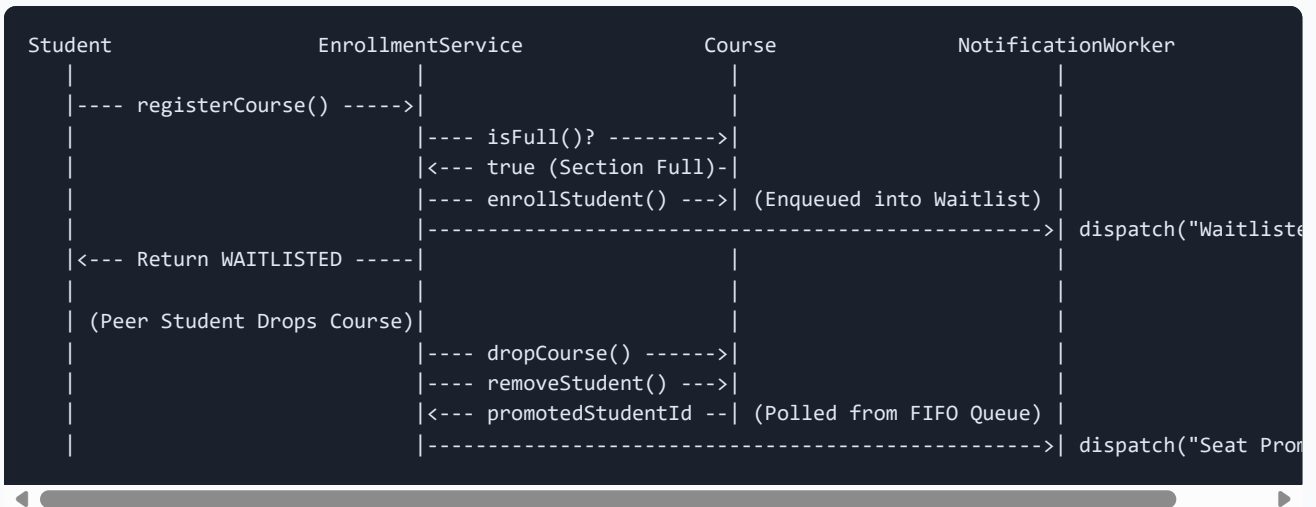
- View Academic Transcript & CGPA
- Calculate Tuition Fees

7.2 Registration Process Flow / Workflow Diagram

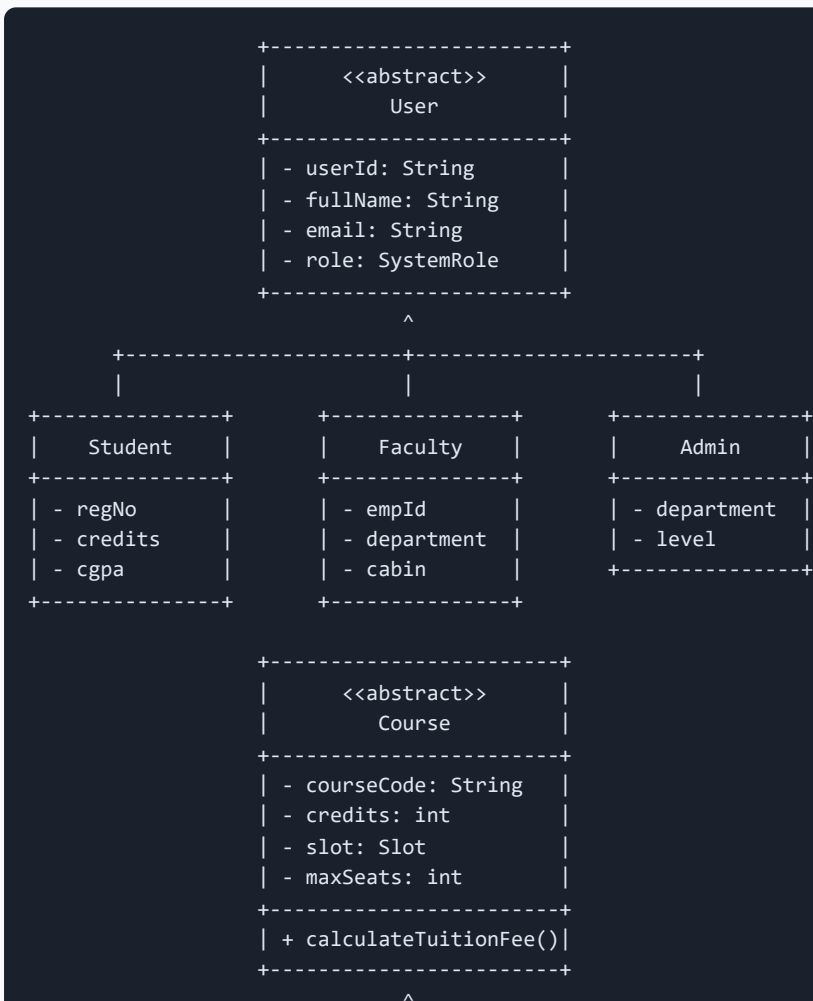
```

[User Initiates Registration]
    |
    v
[Check Student Exists & Active]
    |
    v
[Prerequisites Completed & Passed?] ----(NO)----> [Throw CampusException]
    | (YES)
    v
[Current Credits + Course <= 27?] -----(NO)-----> [Throw CreditLimitExceededException]
    | (YES)
    v
[Timetable Slot Clash Detected?] -----(YES)----> [Throw SlotClashException]
    | (NO)
    v
[Is Course Section Full?]
    /      \
  (NO)    (YES)
    |      |
    v      v
[Set Status:    [Enqueue to FIFO Waitlist]
  ENROLLED]    [Set Status: WAITLISTED]
    |          |
    v          v
[Update Student Credits & Persist]
    |
    v
[Dispatch Asynchronous Notification]
  
```

7.3 Sequence Diagram (Registration & Waitlist Promotion)



7.4 Class / Component Diagram



```

+-----+
|
+-----+
| TheoryCourse |
+-----+
| - lectureHours: int |
| - tutorialHours: int |
+-----+
| + calculateTuitionFee() |
+-----+

+-----+
|
+-----+
| LabCourse |
+-----+
| - practicalHours: int |
| - labEnvironment: String |
| - surcharge: double |
+-----+
| + calculateTuitionFee() |
+-----+

```

7.5 Entity-Relationship (ER) / Storage Diagram

```

+-----+ +-----+ +-----+
| USERS | 1 * | ENROLLMENTS | * 1 | COURSES |
+-----+ +-----+ +-----+
| PK userId | | PK enrollmentId | | PK courseCode |
| fullName | | FK studentRegNo | | courseTitle |
| email | | FK courseCode | | credits |
| role | | status | | slot |
| regNo / empId | | attendedClasses | | maxSeats |
| department | | totalClasses | | facultyEmpId |
+-----+ | grade | +-----+
+-----+ +-----+

```

8. Design Decisions & Rationale

1. **Polymorphic Architecture:** Rather than relying on conditional branching with flags, `TheoryCourse`` and `LabCourse`` extend `Course`` and override `calculateTuitionFee()`, cleanly capturing domain variations.
2. **Custom Domain Exception Hierarchy:** Replacing generic runtime exceptions with domain-specific classes (`SlotClashException``, `CreditLimitExceededException``, `CourseFullException``) provides clear feedback to users.
3. **Repository Pattern with File Synchronization:** The `DataStore`` interface decouples the services from storage mechanisms, allowing seamless transitions between in-memory collections and persistent CSV files.
4. **Asynchronous Daemon Worker:** The `NotificationWorker`` implements `Runnable`` and uses a thread-safe `BlockingQueue`` to process alerts without degrading application responsiveness.

9. Implementation Details

The project is built entirely on standard Java 21 features with zero external runtime dependencies:

- **Encapsulation:** Private fields with explicit accessors/mutators and immutable collections (`Collections.unmodifiableList``).
- **Collections Framework:** `ConcurrentHashMap` for thread safety, `LinkedHashSet` for ordered student rosters, and `LinkedList` for FIFO waitlisting.
- **Regular Expressions:** `InputValidator` validates registration numbers (e.g., 25BAI10611) and course codes (e.g., CSE2006).
- **File I/O (NIO.2):** Utilizes `Files.newBufferedReader()` and `BufferedWriter` for CSV parsing and serialization.

10. Screenshots / Execution Results

10.1 Automated Test Suite Execution

```
=====
SMARTCAMPUS - AUTOMATED VALIDATION TEST SUITE
=====
Candidate: YATHAM JATHINDRA REDDY | Reg No: 25BAI10611 | Course: CSE2006

[PASS] OOP - User Hierarchy & Polymorphism
[PASS] OOP - Course Hierarchy & Polymorphic Tuition Fee
[PASS] Timetable Slot Clash Detection Logic
[PASS] Input Validator Regex Verification
[PASS] Course Registration & Credit Bounds
[PASS] Credit Limit Overload Exception Enforcement
[PASS] Registration Timetable Slot Clash Exception
[PASS] Waitlist Queue & Automatic FIFO Seat Promotion
[PASS] Grading & Dynamic Weighted CGPA Calculation
[PASS] Attendance Tracking & Debarment Alert (<75%)
[PASS] File I/O Persistence & Reload Engine
=====
```

TEST RESULTS: 11/11 PASSED (100.0%)

=====

10.2 Academic Transcript Output

=====

VIT SMARTCAMPUS ACADEMIC TRANSCRIPT

=====

Student Name : YATHAM JATHINDRA REDDY
Registration No: 25BAI10611
Degree Program : B.Tech Computer Science (AI & ML)
Semester : 3
Enrolled Credits: 9 / 27
Cumulative GPA : 9.89

CODE	TITLE	CRED	SLOT	ATTD %	GRADE
CSE2006L	Java Programming Lab	1	L1+L2	90.0%	A
CSE2006	Programming in Java	4	A1	90.0%	S
CSE3999	Special Topics in Deep Lear...	2	E1	100.0%	IP
CSE2001	Data Structures and Algorithms	4	B1	66.7%	S

=====

11. Testing Approach

Quality assurance was achieved through automated unit testing and full system demonstration:

- **Unit Testing (SystemTestSuite):** Contains 11 tests validating inheritance, slot clash detection, credit overloads, waitlist promotions, grading calculations, and CSV persistence.
- **Automated Demonstration (Main --demo):** Executes an end-to-end integration walkthrough covering all user workflows without requiring manual input.

12. Challenges Faced

- **Compound Slot Clashes:** Handling composite laboratory slots (e.g. L1+L2) required tokenization and set overlap algorithms.
- **Atomic Waitlist Promotion:** Preventing race conditions during simultaneous drops and adds required thread synchronization.
- **Incomplete Courses in GPA:** Ungraded courses were excluded from GPA weight calculations to prevent skewed semester results.

13. Learnings & Key Takeaways

- Practical application of core OOP principles in modeling real-world academic constraints.
- Effective use of the Java Collections Framework (Sets, Maps, FIFO Queues).
- Implementation of multithreaded background processing using standard Java concurrency utilities.
- Building resilient systems with custom exceptions and defensive programming techniques.

14. Future Enhancements

- **Spring Boot REST APIs:** Exposing backend endpoints via Spring Web with JWT security.
- **Relational Database:** Migrating CSV storage to PostgreSQL using Spring Data JPA.
- **Interactive Web Portal:** Developing a responsive React/Angular interface with timetable grid rendering.

15. References

1. Schildt, Herbert. *Java: The Complete Reference*, 12th Edition. McGraw-Hill Education, 2021.
2. Bloch, Joshua. *Effective Java*, 3rd Edition. Addison-Wesley Professional, 2018.
3. Oracle Corporation. *Java SE 21 Documentation and Language Specifications*, 2023.
4. Vellore Institute of Technology. *FFCS Academic Regulations Handbook*, 2025.